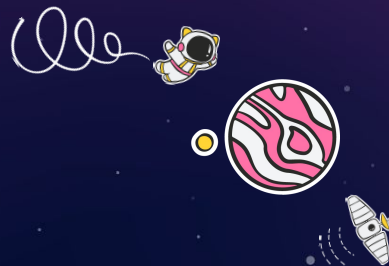


2026
odoo

EXPERIENCE

Into the Odoo Editor

Aka ``html_editor``



*This is a **basic introduction**
Refer to the actual documentation
for full details*

*Scan to ask your
questions*

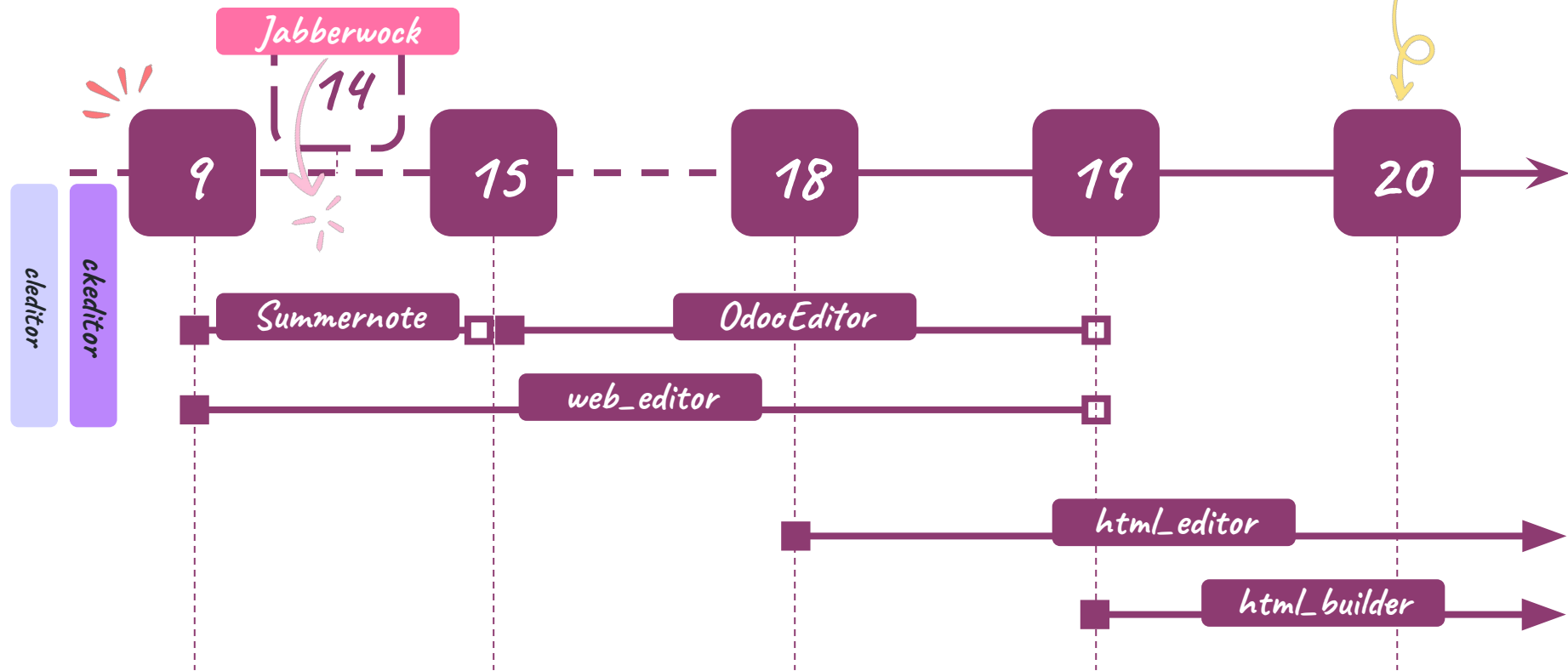


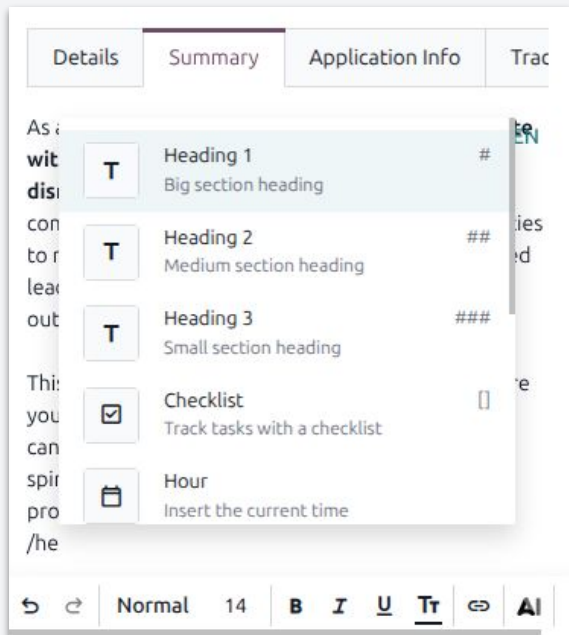
Benoit Socias, Research & Development

<https://pad.odoo.com/p/oxp26-editor>

A bit of history

We are here





*Where is
the Editor?*

Nowhere and everywhere



your own app?



knowledge

HTML fields



website



mass_mailing



studio reports

html_builder

html_editor



public forum





Editors are made of

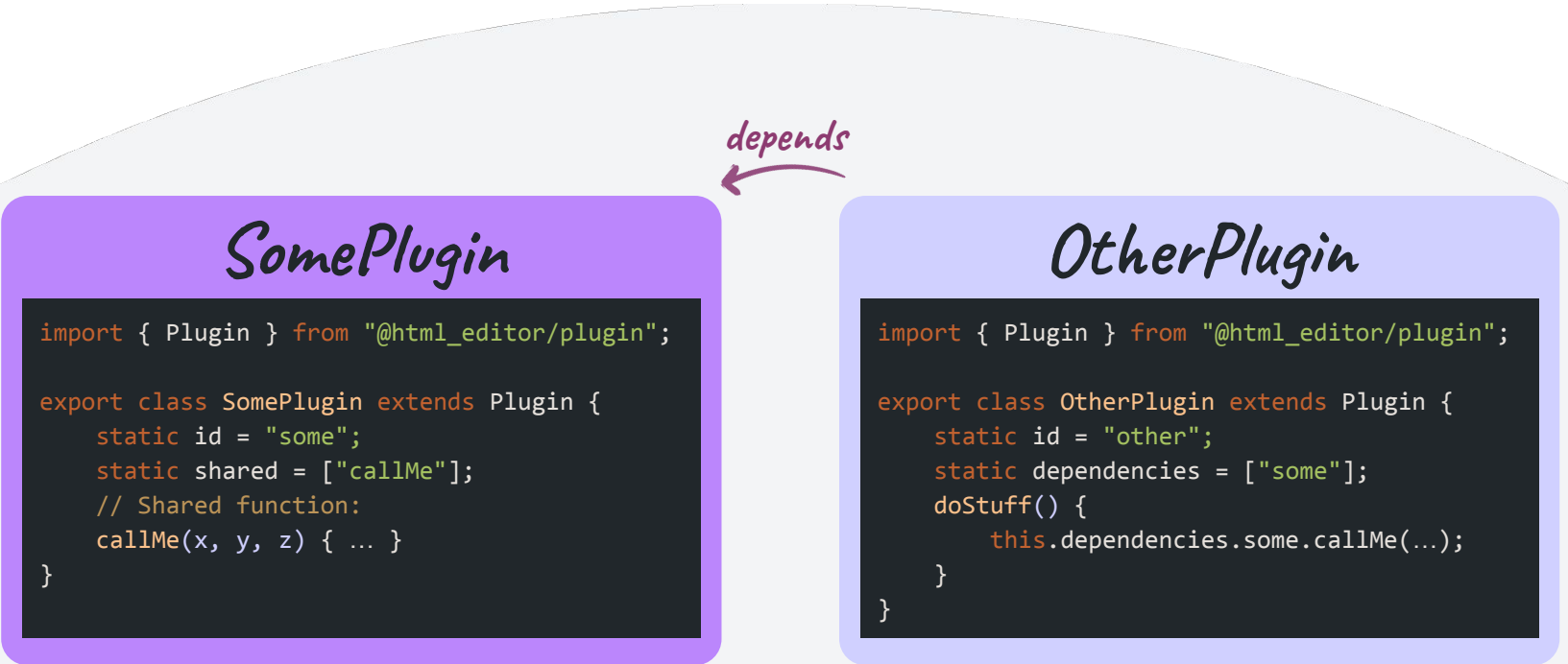
Plugins



Do not mix with
Owl 3 Plugins
which replace
Owl 2 Services

Plugins

depends



```
graph TD
    subgraph EditorConfiguration
        direction LR
        subgraph SomePluginBox [SomePlugin]
            SP["import { Plugin } from '@html_editor/plugin';\n\nexport class SomePlugin extends Plugin {\n  static id = 'some';\n  static shared = ['callMe'];\n  // Shared function:\n  callMe(x, y, z) { ... }\n}"]
        subgraph OtherPluginBox [OtherPlugin]
            OP["import { Plugin } from '@html_editor/plugin';\n\nexport class OtherPlugin extends Plugin {\n  static id = 'other';\n  static dependencies = ['some'];\n  doStuff() {\n    this.dependencies.some.callMe(...);\n  }\n}"]
        OP -- depends --> SP
    end
```

SomePlugin

```
import { Plugin } from "@html_editor/plugin";

export class SomePlugin extends Plugin {
  static id = "some";
  static shared = ["callMe"];
  // Shared function:
  callMe(x, y, z) { ... }
}
```

OtherPlugin

```
import { Plugin } from "@html_editor/plugin";

export class OtherPlugin extends Plugin {
  static id = "other";
  static dependencies = ["some"];
  doStuff() {
    this.dependencies.some.callMe(...);
  }
}
```

Editor Configuration

Plugins Sets

Core

- Input
- Selection
- Delete
- Dom
- Format
- Style
- LineBreak
- History

Main

- Color
- List
- Table
- Media
- ShortCut
- PowerBox
- Toolbar
- Link

Collaboration

Embedded Component 

Builder

(Website, Mass mailing)

... yours ?

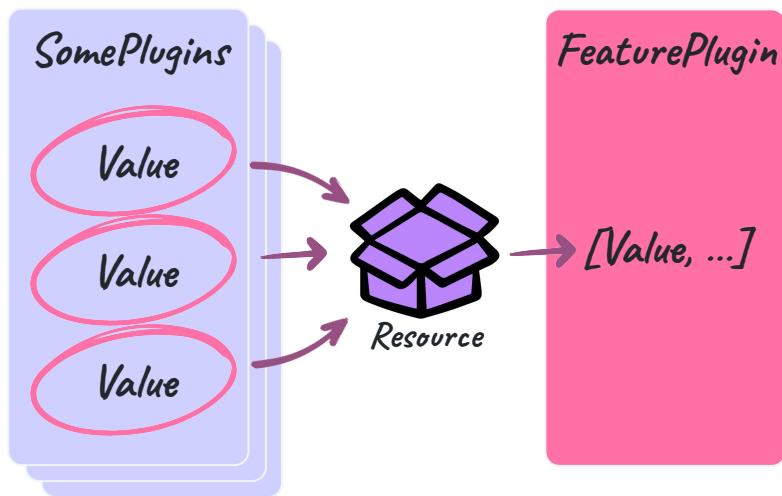


Plugins Resources

Participate in some mechanism

General

- getResources(...)



```
import { Plugin } from "@html_editor/plugin";
import { withSequence }
  from "@html_editor/utils/resource";

export class MyPlugin extends Plugin {
  someCode() {
    const values = this.getResources("someResource");
    // values = ["XYZ", "A", "B", "C"]
  }
}

export class AbcPlugin extends Plugin {
  resources = {
    someResource: ["A", "B", "C"],
  };
}

export class XyzPlugin extends Plugin {
  resources = {
    someResource: withSequence(5, "XYZ"),
  };
  // default is 10
}
```

Integrating into the Editor



Powerbox

Toolbar

FORMAT

T	Heading 1 Big section heading	#
T	Heading 2 Medium section heading	##
T	Heading 3 Small section heading	###
T	Text	

Normal 14 B I U Tt ↺ AI ⋮

⋮ Action on selected text inside the toolbar

↶ Switch the text's direction

<> Code

Integrating

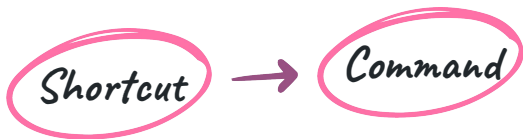
Toolbar

Command

```
resources = {  
  user_commands: [{  
    id: "someCommand",  
    description: _t("Do stuff"),  
    icon: "question_mark", // from material symbols  
    run: (commandParams) => ...,  
    isAvailable: (selection) => ...,  
  }],  
  shortcuts: [{  
    hotkey: "control+?", commandId: "someCommand",  
  }],  
  toolbar_groups: [{ id: "someGroup" }],  
  toolbar_items: [{  
    id: "someToolbarButton",  
    description: t_("Do Stuff (Ctrl + ?)"),  
    groupId: "someGroup",  
    namespaces: ["compact", "extended"],  
    commandId: "someCommand",  
    isActive: (selection, nodes) => ...,  
    isDisabled: (selection, nodes) => ...,  
  }],  
};
```

Integrating

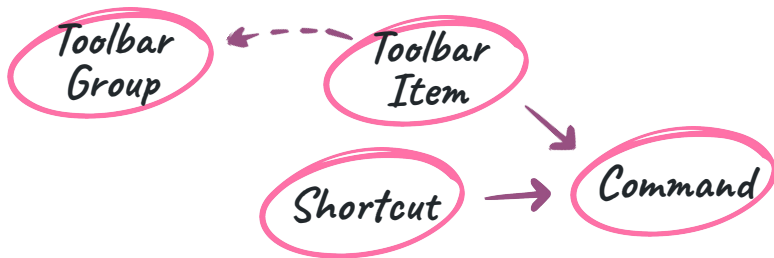
Toolbar



```
resources = {  
  user_commands: [{  
    id: "someCommand",  
    description: _t("Do stuff"),  
    icon: "question_mark", // from material symbols  
    run: (commandParams) => ...,  
    isAvailable: (selection) => ...,  
  }],  
  shortcuts: [{  
    hotkey: "control+?", commandId: "someCommand",  
  }],  
  toolbar_groups: [{ id: "someGroup" }],  
  toolbar_items: [{  
    id: "someToolbarButton",  
    description: t_("Do Stuff (Ctrl + ?)"),  
    groupId: "someGroup",  
    namespaces: ["compact", "extended"],  
    commandId: "someCommand",  
    isActive: (selection, nodes) => ...,  
    isDisabled: (selection, nodes) => ...,  
  }],  
};
```

Integrating

Toolbar



Namespaces

- compact



- extended



- image



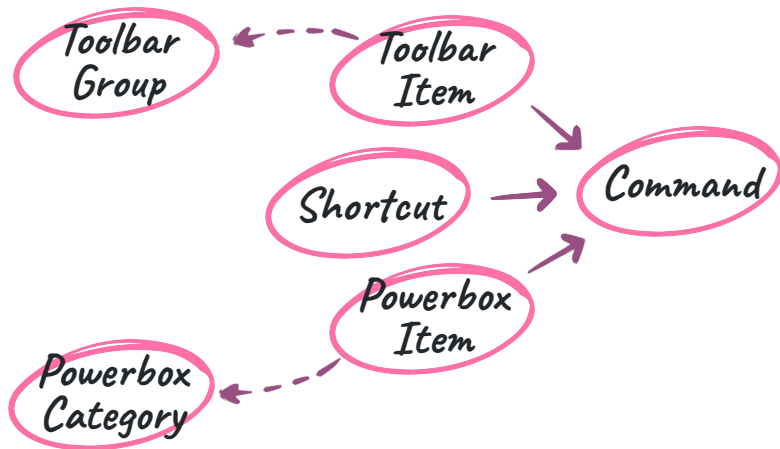
- table



```
resources = {
  user_commands: [{
    id: "someCommand",
    description: _t("Do stuff"),
    icon: "question_mark", // from material symbols
    run: (commandParams) => ...,
    isAvailable: (selection) => ...,
  }],
  shortcuts: [{
    hotkey: "control+?", commandId: "someCommand",
  }],
  toolbar_groups: [{ id: "someGroup" }],
  toolbar_items: [{
    id: "someToolbarButton",
    description: t_("Do Stuff (Ctrl + ?)"),
    groupId: "someGroup",
    namespaces: ["compact", "extended"],
    commandId: "someCommand",
    isActive: (selection, nodes) => ...,
    isDisabled: (selection, nodes) => ...,
  }],
};
```

Integrating

PowerBox



```
resources = {
  user_commands: [{
    id: "someCommand",
    description: _t("Do stuff"),
    icon: "question_mark", // from material symbols
    run: (commandParams) => ...,
    isAvailable: (selection) => ...,
  }],
  ...
  powerbox_categories: [{
    id: "someCategory",
    name: _t("Some Category"),
  }],
  powerbox_items: [{
    title: _t("Do Stuff"),
    description: _t("Does stuff on selected text"),
    icon: "question_mark", // from material symbols
    categoryId: "someCategory",
    commandId: "someCommand",
    keywords: [_t("thing")],
  }],
};
```

Integrating

Selection



Major Tools

- Inspect current selection
 - `getSelectionData()`
- Remember and restore selection
 - `preserveSelection()`
 - `.restore()`

```
import { Plugin } from "@html_editor/plugin";

export class SomePlugin extends Plugin {
  static dependencies = ["selection"];

  someCode() {
    const selectionData = this.dependencies.selection
      .getSelectionData();
    const selection = selectionData
      .deepEditableSelection;
    if (selection.isCollapsed) {
      return;
    }

    const cursor = this.dependencies.selection
      .preserveSelection();
    // DOM changes that touch the selection
    cursor.restore();
  }
}
```

Integrating

History

General

- Commits
 - `commit()`
- Undo / Redo
 - `canUndo()`
 - `undo()`
 - `canRedo()`
 - `redo()`

Tools

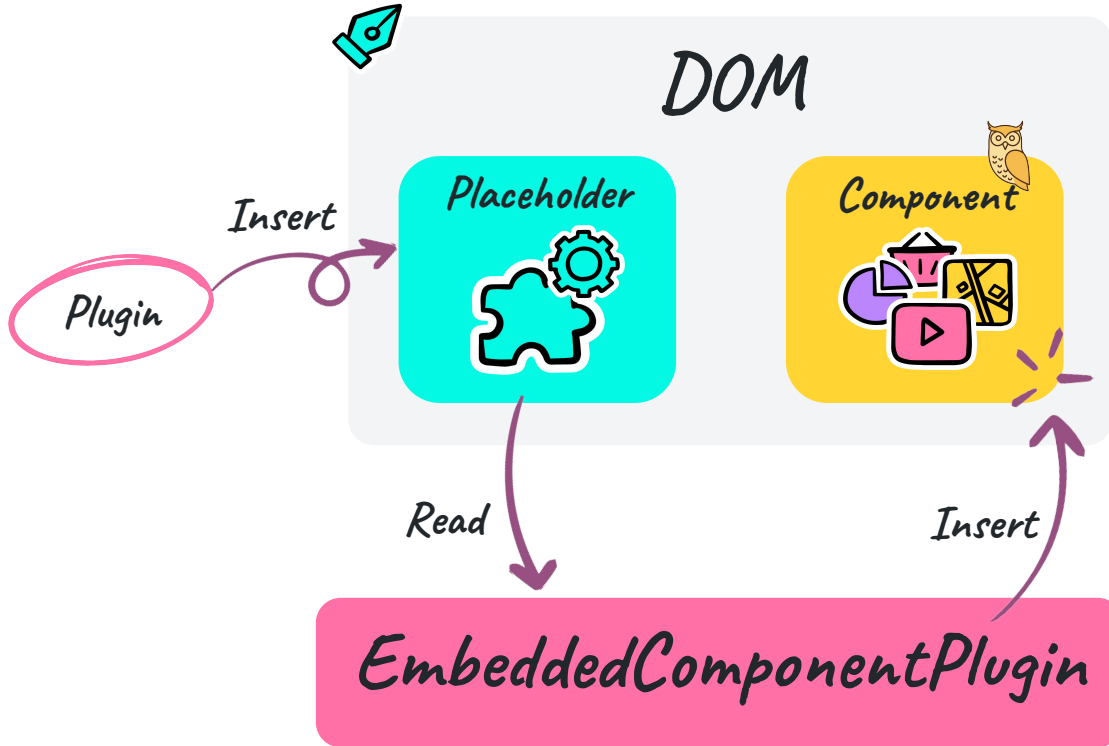
- Save points
 - `makeSavePoint()`
 - `.revert()`

```
import { Plugin } from "@html_editor/plugin";

export class SomePlugin extends Plugin {
  static dependencies = ["history"];

  someCode() {
    const revert =
      this.dependencies.history.makeSavePoint();
    const success = trySomeStuff();
    if (!success) {
      revert();
      doSomethingElse();
    }
    this.dependencies.history.commit();
  }
}
```

Embedded Components



Inside Website
public pages Colibri
interactions must be
used instead

Plugins Resources

Participate in some mechanism

General

- `getResources(...)`



resources values can be functions
 $(...) \Rightarrow \dots$



Plugins Resources

Participate in some mechanism

General

- getResources(...)

Tools

- trigger(...)
- triggerAsync(...)
- delegateTo(...)
- processThrough(...)
- checkPredicates(...)

→ <https://github.com/odoo/odoo/pull/236109>

Event handling



```
import { Plugin } from "@html_editor/plugin";

export class SomeMechanismPlugin extends Plugin {
  async someCode() {
    const values = this.trigger(
      "on_some_stuff_handlers", 1, 2);
    // values = [3]
    await this.triggerAsync(
      "on_some_stuff_async_handlers", 3, 4);
  }
}

export class ParticipatingPlugin extends Plugin {
  resources = {
    on_some_stuff_handlers: (param1, param2) => {
      return param1 + param2;
    },
    on_some_stuff_async_handlers: async (p1, p2) => {
      return someNetworkCall(p1, p2);
    },
  };
}
```

Plugins Resources

Participate in some mechanism

General

- getResources(...)

Tools

- trigger(...)
- triggerAsync(...)
- delegateTo(...)
- processThrough(...)
- checkPredicates(...)

→ <https://github.com/odoo/odoo/pull/236109>

Delegation



```
import { Plugin } from "@html_editor/plugin";

export class SomeMechanismPlugin extends Plugin {
  someCode() {
    if (this.delegateTo(
      "some_stuff_overrides", 1, 2)) {
      return;
    }
    // default behavior
  }
}

export class ParticipatingPlugin extends Plugin {
  resources = {
    some_stuff_overrides: (param1, param2) => {
      if (this.isMyStuff(param1)) {
        // specific behavior
        return true;
      }
    },
  };
}
```

Plugins Resources

Participate in some mechanism

General

- getResources(...)

Tools

- trigger(...)
- triggerAsync(...)
- delegateTo(...)
- processThrough(...)
- checkPredicates(...)

→ <https://github.com/odoo/odoo/pull/236109>

Process chain



```
import { Plugin } from "@html_editor/plugin";

export class SomeMechanismPlugin extends Plugin {
  someCode(item) {
    const processedItem = this.processThrough(
      "some_stuff_processors", item, 1, 2);
    // use processed item
  }
}

export class ParticipatingPlugin extends Plugin {
  resources = {
    some_stuff_processors: (item, p1, p2) => {
      return {
        ...item,
        myImpact: someOperation(item, p1, p2),
      };
    },
  };
};
```

Plugins Resources

Participate in some mechanism

General

- getResources(...)

Tools

- trigger(...)
- triggerAsync(...)
- delegateTo(...)
- processThrough(...)
- checkPredicates(...)

→ <https://github.com/odoo/odoo/pull/236109>

Deciding



```
import { Plugin } from "@html_editor/plugin";

export class SomeMechanismPlugin extends Plugin {
  someCode(item) {
    const condition = this.checkPredicate(
      "some_stuff_predicates", 1, 2) ?? true;
    // do not forget default value
  }
}

export class ParticipatingPlugin extends Plugin {
  resources = {
    some_stuff_predicates: (param1, param2) => {
      if (this.isMyConcern(param1)) {
        return param2 > 0;
      }
      // undefined results are ignored
    },
  };
}
```



The eFEFFant
in the room

FEFF explanation

The problem

Where is the cursor?

- Cursor is |before the link
- Cursor is inside the link
- Cursor is inside| the link
- Cursor is after| the link

?



The FEFF approach

Zero-width space character

- Cursor is |before the link
- Cursor is inside the link
- Cursor is inside| the link
- Cursor is after| the link



Unit testing with

Hoot

Hoot tests

Unit testing



Tools

- Two types of tests
 - testEditor(...)
 - setupEditor(...)
- Selection representation:
 - some **[selected]** text

anchor  *focus* 

- some cursor  position

collapsed 

```
import { test } from "@odoo/hoot";
import { setupEditor, testEditor }
  from "@html_editor/./tests/_helpers/editor";
```

```
test("simplifiedTest", async () => {
  await testEditor({
    contentBefore: `<div>a[b]c</div>`,
    stepFunction: (editor) => ...,
    contentAfter: `<div>ax[]c</div>`,
    ...someConfig,
  });
});

test("classicHootTest", async () => {
  await setupEditor(`<p>a[b]c</p>`, { ...someConfig });
  // usual Hoot assertions
});
```

Summary

- Explore existing **Shared** methods and **Resources**
- Define your own **Plugins** with the **Dependencies** you need
- Register them inside **Plugin sets**

Q&A



<https://pad.odoo.com/p/oxp26-editor>

Annexes





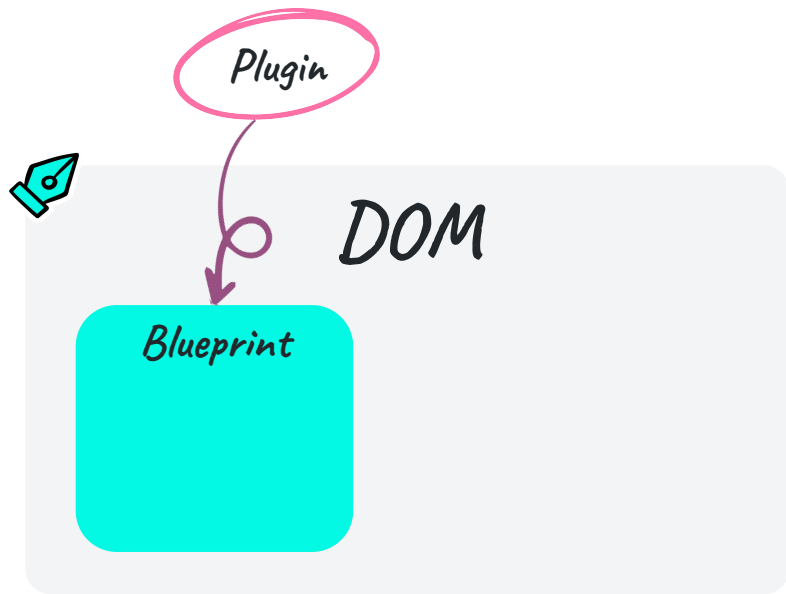
Enabling dynamic content

Embedded Components



Embedded Components

Inserting



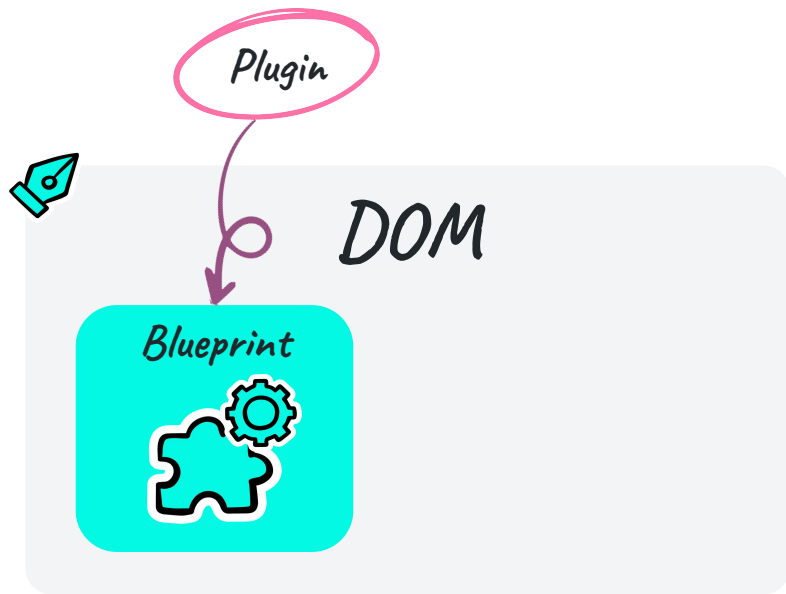
```
import { Plugin } from "@html_editor/plugin";
import { renderToElement } from "@web/core/utils/render";

export class SomePlugin extends Plugin {
  static dependencies = [
    "dom", "history", "selection"];

  insertMyComponent(props) {
    const embedEl = renderToElement(
      "myapp.my_blueprint", {
        embedded_props: JSON.stringify(props),
      });
    this.dependencies.dom.insert(embedEl);
    this.dependencies.history.commit();
    this.dependencies.selection.setSelection({
      anchorNode: ..., anchorOffset: ...});
  }
}
```

Embedded Components

Blueprint

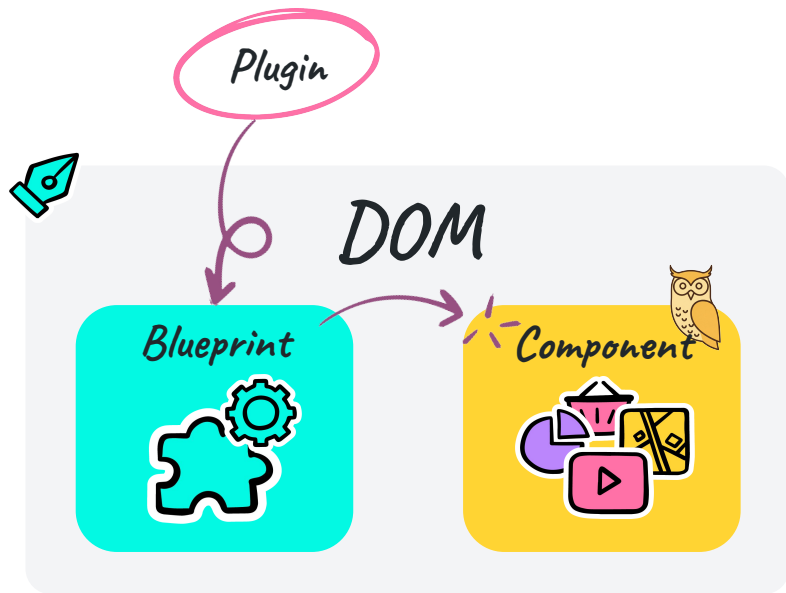


```
<!-- In /myapp/static/.../my_blueprint.xml -->
```

```
<t t-name="myapp.my_blueprint">  
  <div  
    t-att-data-embedded-props="embeddedProps"  
    data-embedded="my_embedded_component"  
    data-oe-protected="true"  
    contenteditable="false"  
    class="o-contenteditable-false"  
  />  
</t>
```

Embedded Components

Registering



```
import { Component, props, types as t } from "@odoo/owl";
import { getEmbeddedProps
    }
    from "@html_editor/others/embedded_component_utils";

export class SomeEmbeddedComponent extends Component {
    static template = "myapp.my_embedded_template";
    props = props({
        myProp: t.string(),
    });

}

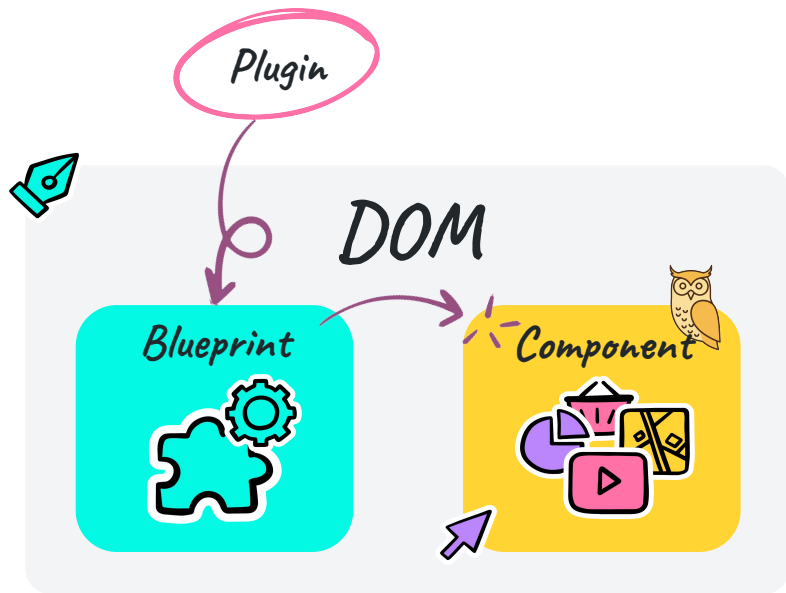
export const myEmbedding = {
    name: "my_embedded_component",
    Component: SomeEmbeddedComponent,
    getProps: host => ({host, ...getEmbeddedProps(host)}),

};
```

add to an embedding set

Embedded Components

Editing



```
import { Component, props, types as t } from "@odoo/owl";
import { getEmbeddedProps, StateChangeManager,
        useEmbeddedState }
    from "@html_editor/others/embedded_component_utils";

export class SomeEmbeddedComponent extends Component {
    static template = "myapp.my_embedded_template";
    props = props({
        host: t.object(),
        myProp: t.string(),
    });
    setup() {
        this.state = useEmbeddedState(this.props.host);
    }
}

export const myEmbedding = {
    name: "my_embedded_component",
    Component: SomeEmbeddedComponent,
    getProps: host => ({host, ...getEmbeddedProps(host)}),
    getStateChangeManager:
        config => new StateChangeManager(config),
};
```

Thank you!



#OdooExperience



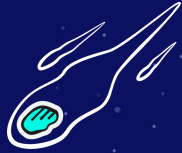
/odoo



@odoo.official



@odoo



odoo²⁰²⁶

EXPERIENCE